

# Coding and Solution

PocketSmart AI | Team ID 6aba1a835015c35b98635933 | Team Leader: Rohith S | Members: Heedhesh R, Ashwin S, Chandru T

---

The application source code is in this folder (see README.md for setup). This file summarises how the solution is organised.

## Code Layout

```
5. Project Development Phase/
■■■ app.py                # FastAPI app and shared routes
■■■ auth.py               # Password hashing and JWT verification
■■■ database.py           # SQLite schema and persistence
■■■ models.py             # Pydantic user and token models
■■■ planner_service.py    # Shared plan creation and persistence
■■■ recommendations.py    # Gemini provider and offline fallback
■■■ routers/              # auth, home, party, jewelry routes
■■■ templates/            # Jinja2 pages and inline browser scripts
■■■ static/               # CSS and favicon
■■■ requirements.txt
■■■ .env.example
```

## Functional Features Included

1. Registration, login and logout with bcrypt and JWT cookies
2. Dashboard linking to all planners
3. Home Interior Planner
4. Party Budget Planner
5. Jewelry Planner with optional outfit image
6. Gemini integration with model fallback (gemini-2.5-flash, gemini-2.0-flash)
7. Offline rule-based plans
8. Per-user history with clear option
9. Input validation and output escaping
10. Automated test suite

## Readability and Reusability

- Routes are separated by feature, and plan creation is shared through `planner_service.py`.
- All planners call one `generate_recommendations(category, data)` function.
- Budget splitting is reused through a single `_split_budget` helper.
- Configuration comes from environment variables, not constants in code.